

Autoostas projekta dokumentācija

Prasības, modeļi, datu bāze, interfeiss, implementācija un testēšana.

Projekts	Autoostas informācijas sistēma
Datums	2026-05-12
Tehnoloģijas	HTML, CSS, JavaScript, Python, SQLite

Autoostas informācijas sistēmas projekta dokumentācija

1. Projekta mērķis

Izstrādāt tīmekļa lietotni autoostai, kurā pasažieri var meklēt autobusu maršrutus un pirkt biļetes, autobusa vadītāji var veidot maršrutus, bet autoostas vadītājs var pārvaldīt sistēmas datus un skatīt pārskatus.

2. Lietotāju grupas

Lietotājs	Apraksts	Galvenās tiesības
Viesis	Lietotājs bez konta	Meklēt un apskatīt maršrutus, atvērt pieteikšanos vai konta izveidi
Pasažieris	Reģistrēts lietotājs	Papildināt bilanci, pirkt biļetes, saņemt QR kodu un čeku
Autobusa vadītājs	Lietotājs ar šofera statusu	Veidot un labot savus maršrutus
Autoostas vadītājs	Autobusa vadītājs ar pārvaldības tiesībām	Redzēt visus lietotājus, maršrutus, iemaksas, biļetes un pārskatus

3. Funkcionālās prasības

ID	Prasība
F-01	Sistēmai jāļauj ievadīt sākumpunktu un galapunktu maršrutu meklēšanai.
F-02	Sistēmai jāparāda piemēroti maršruti pēc meklēšanas.
F-03	Sistēmai jānodrošina pieteikšanās ar konta nosaukumu un paroli.
F-04	Pieteikšanās logā parolei jābūt paslēpjamai un parādāmai ar izvēles rūtiņu.
F-05	Sistēmai jāļauj izveidot kontu ar vārdu, uzvārdu, vecumu, konta nosaukumu un paroli.
F-06	Parolei jābūt vismaz 8 simbolus garai, ar lielo burtu, ciparu un speciālo simbolu.

ID	Prasība
F-07	Konta nosaukumi nedrīkst atkārtoties.
F-08	Pēc pieteikšanās lietotājam jāredz vārds, uzvārds, bilance un loma.
F-09	Lietotājs var papildināt bilanci par summu virs 0 un līdz 200 EUR.
F-10	Lietotājs var redzēt iepriekšējos biļešu pirkumus.
F-11	Lietotājs var izvēlēties reisu, redzēt cenu un apmaksāt biļeti no bilances.
F-12	Ja bilancē nepietiek līdzekļu, sistēmai jāparāda kļūda.
F-13	Pēc apmaksas sistēmai jāparāda QR kods un lejupielādējams čeks.
F-14	Lietotājs var pieteikties par autobusa vadītāju.
F-15	Ja lietotāja vecums mīnus šofera pieredze ir mazāks par 18, šofera pieteikums netiek pieņemts.
F-16	Autobusa vadītājs var veidot maršrutu atsevišķā lapā.
F-17	Maršrutam var norādīt sākumu, galapunktu, cenu un pieturvietas.
F-18	Pieturvietas var pievienot un noņemt ar plus/mīnus pogām.
F-19	Vienam maršrutam var pievienot vairākus manuālus reisa laikus.
F-20	Vienam maršrutam var ģenerēt atkārtotošus laikus pēc intervāla un laika robežām.
F-21	Autobusa vadītājs var kļūt par autoostas vadītāju.
F-22	Autoostas vadītājs var pārvaldības lapā redzēt lietotājus, maršrutus, iemaksas un biļetes.
F-23	Autoostas vadītājs var skatīt pārskatus: visvairāk iztērēts, populārākais maršruts, lielākā iemaksa un kopējie ienākumi.
F-24	Visi dati jāglabā SQL datu bāzē.

4. Nefunkcionālās prasības

ID	Prasība
N-01	Lietotāja saskarnei jābūt latviešu valodā.
N-02	Kļūdu paziņojumiem jābūt redzamiem un saprotamiem.

ID	Prasība
N-03	Dialoglogu fons nedrīkst būt tik izpludināts, ka kļūdas nav salasāmas.
N-04	Paroles nedrīkst glabāties atklātā tekstā.
N-05	Datu bāzei jāizmanto ārējās atslēgas un primārās atslēgas.
N-06	Sistēmai jābūt palaīžamai lokāli ar Python serveri.
N-07	Sistēmai jābūt pārbaudāmai ar automātiskiem testiem.

5. Sistēmas modelis

Sistēma sastāv no trim slāņiem:

Slānis	Faili	Atbildība
Lietotāja interfeiss	index.html, route.html, manager.html, styles.css	Skati, formas, dialoglogi un lapu izkārtojums
Klienta loģika	app.js, route-page.js, manager.js	Lietotāja darbības, datu ielāde, validācija, QR koda un čeka veidošana
Serveris un datu bāze	server.py, database/schema.sql, autoosta.db	API, SQL glabāšana, paroles hash, pirkumu un iemaksu transakcijas

Galvenās datu plūsmas:

1. Viesis ievada sākumu/galapunktu, klients ielādē maršrutus no API un parāda rezultātus.
2. Lietotājs reģistrējas vai piesakās, serveris pārbauda datus un atgriež lietotāja publisko profilu.
3. Lietotājs papildina bilanci, serveris pārbauda summu un saglabā iemaksu.
4. Lietotājs pērk biļeti, serveris pārbauda bilanci, samazina to un saglabā pirkumu.
5. Šoferis veido maršrutu, serveris saglabā maršrutu, pieturas un reisa laikus.
6. Autoostas vadītājs atver pārvaldības lapu un redz apkopotos datus.

6. Datu bāzes projektējums

SQL datu bāze ir SQLite fails autoosta.db. Shēma atrodas database/schema.sql.

Tabula	Nozīme	Galvenie lauki
users	Lietotāju konti un lomas	username, first_name, last_name, age, password_hash, balance, is_driver, is_station_manager
driver_applications	Šoferu pieteikumi	id, username, license_number, experience_years, motivation
routes	Maršrutu pamatdati	id, name, start_point, end_point, price, driver_username
route_stops	Maršruta pieturvietas	id, route_id, stop_name, sequence_number

Tabula	Nozīme	Galvenie lauki
route_schedules	Reisa laiki	id, route_id, departure, arrival, sequence_number
purchases	Nopirktās biļetes	id, ticket_number, username, route_id, price, paid, created_at
top_ups	Bilances iemaksas	id, username, amount, balance_after, created_at

Datu bāze ir normalizēta: lietotāji, maršruti, pieturas, reisa laiki un pirkumi tiek glabāti atsevišķās tabulās. Daudzvērtību dati, piemēram, pieturvietas un reisa laiki, nav glabāti vienā teksta laukā, bet atsevišķās saistītās tabulās.

7. Realitāšu-saišu modelis

ER modelis ir sagatavots failā autoostas-er-diagramma.md. Galvenās saites:

- viens lietotājs var veikt daudz bilances iemaksu;
- viens lietotājs var nopirkt daudz biļešu;
- viens šoferis var izveidot daudz maršrutu;
- vienam maršrutam var būt daudz pieturvietu;
- vienam maršrutam var būt daudz reisa laiku;
- viena biļete attiecas uz vienu lietotāju un vienu maršrutu.

8. Lietotāja interfeisa projektējums

Wireframe ir aprakstīts failā lietotāja-interfeisa-wireframe.md. Sistēmai ir šādi galvenie skati:

Skats	Lietotājs	Saturs
Sāukmlapa	Viesis/pasažieris	Maršrutu meklēšana, pieteikšanās, konta izveide
Pieteikšanās dialogs	Viesis	Konta nosaukums, parole, paroles redzamības rūtiņa
Konta izveides dialogs	Viesis	Vārds, uzvārds, vecums, lietotājvārds, parole divas reizes
Bilances dialogs	Pasažieris	Bilance, papildināšana, iepriekšējās iemaksas
Biļetes rezultāts	Pasažieris	QR kods un čeka lejupielāde
Maršruta izveides lapa	Šoferis	Maršruta dati, pieturas, reisa laiki
Pārvaldības lapa	Autoostas vadītājs	Lietotāji, maršruti, iemaksas, biļetes, pārskati

9. Implementācija

Projekts ir realizēts ar HTML, CSS, JavaScript, Python un SQLite. Python serveris apkalpo statiskos failus un API pieprasījumus. JavaScript faili nodrošina formas, validācijas, datu ielādi un lietotāja darbības.

Svarīgākie uzlabojumi:

- paroles tiek glabātas kā sālīts PBKDF2 hash;
- paroles hash netiek atdots klienta pusei;
- reģistrācija un pieteikšanās tiek pārbaudīta servera pusē;
- bilances papildināšana un biļetes pirkšana notiek serverī vienā datu bāzes darbībā;
- SQLite savienojumi tiek korekti aizvērti;
- pievienoti automātiskie testi servera biznesa noteikumiem.

10. Testēšana

Testu plāns atrodas failā testu-plans.md. Automātiskie testi atrodas mapē tests.

Pārbaudītās jomas:

- paroles drošība;
- unikāli lietotājevārdi;
- šoferu vecuma/pieredzes pārbaude;
- bilances papildināšanas limits;
- biļetes pirkšanas transakcija.

Automātiskie testi palaisti ar komandu:

```
python -m unittest discover -s tests -v
```

Rezultāts: 5 testi izpildījās veiksmīgi.

11. Palaišana

No projekta mapes jāpalaiž:

```
python server.py 8001
```

Pārlūkā jāatver:

```
http://127.0.0.1:8001/
```

Pēc noklusējuma sistēmā ir tehniskais lietotājs sistema, kas tiek izmantots sākotnējiem maršrutiem. Tā parole ir Sistema!1.